RINEYA RESEARCH / RCWM-001

Actions have consequences.

A consequence world model for compact language intelligence

Research and implementation plan

Typed action representations. Learned consequence embeddings.
Goal-conditioned utility. Long-horizon and parallel decision-making.

The central test
Can a learned consequence representation improve real action selection or reduce the labels needed for new goals, compared with a matched scalar evaluator?

Prepared for Cheng Li

5 October 2026 / Research edition 1.0

43 primary-source references. A gated experimental program.
Editable specifications, schemas, configurations and tested CPU reference code.
No newly trained or qualified native Rineya model is claimed.

Contents

1 Executive decision 2

2 Scope, provenance and evidence discipline 3

3 Formal model: actions, consequences and goals 4

4 Relationship to prior art and the defensible contribution 6

5 Architecture and implementation boundaries 7

6 Executable actions and consequence semantics 9

7 Learning the consequence space and testing clustering 10

8 Information flow, masking and counterfactual validity 11

9 Data program and environment ladder 12

10 Training recipe and staged objectives 14

11 Planning, uncertainty and real-environment feedback 15

12 Long context, delayed credit and parallel task structure 17

13 Experimental program and promotion gates 18

14 Resource engineering for an 8 GB laptop 20

15 Preservation and integration with existing Rineya work 21

16 Engineering, safety and release controls 22

17 Delivery roadmap and ownership 23

18 Risk register and diagnostic responses 24

19 Future extensions and research questions 24

20 How to use the delivered resources 25

21 Glossary and notation 26

22 Delivered reference mechanics and validation 27

23 Source register 28

1 Executive decision

Build a consequence-conditioned decision model for the exact Rineya 0.6B-class checkpoint. Start with a small, independently testable predictor attached to frozen Rineya representations; advance to typed consequence and action slots inside Rineya only when the predictive representation improves real decisions.

The motivating idea is to give actions a learned meaning in terms of what they cause. Instead of training a network only to output a scalar approval score, train it to predict a reusable representation of consequences. A separate goal readout determines which consequences are valuable. Attention can relate a proposed action to distant context, prerequisites, concurrent work and delayed outcomes. This is the user's central proposal, formalized here as **RCWM-001: Rineya Consequence World Model**. Earlier working names such as reward-world-model and reward-token Transformer refer to this same research line, not additional projects. [I03]

The research objective is not to demonstrate that numerical rewards can be embedded. It is to demonstrate that **learning and reusing consequence structure gives a better accuracy–cost–adaptation trade-off than a comparably powerful scalar evaluator**. The most persuasive result would be better action selection on new goals or unseen compositions while holding environment experience, model capacity, candidate plans and evaluation compute constant.

1.1 The first decisive experiment

Use a deterministic text/symbolic environment with exact state snapshots, executable actions and an independent outcome checker. Collect multiple action branches from the same starting state. Freeze the Rineya language backbone. Compare a matched scalar Transformer, a measured-feature predictor and a learned-consequence predictor. Hold out complete problem families and goal combinations. Ask whether the learned predictor chooses better actions using fewer new reward labels, not whether its embedding plot looks attractive.

A successful first milestone is deliberately narrow: **on held-out parents, the consequence representation supports better goal-conditioned decisions at a measured cost and without changing the frozen language model.** A null result is informative. It may show that the task lacks reward diversity, the representation is insufficient, the proposed actions are poor, or scalar prediction is already adequate.

1.2 What this program is, and is not

This is a research plan with an executable synthetic reference scaffold, not a delivered trained Rineya world model. No production checkpoint is included. No new Rineya capability gain, native GPU qualification, remote branch, paid job or external account action was performed for this bundle. The reference code checks selected mechanics in a small CPU environment. It does not implement all later research arms.

World-model learning and reinforcement learning are overlapping components, not competing taxonomies. A model of immediate rewards is a reward model; a model of policy-conditioned cumulative rewards is a value model. A predictor becomes a useful world model for this project when its action-conditioned consequences correspond to independently observable outcomes and improve control. Modeling Rineya's hidden-state changes alone is an internal computation model, which must be evaluated separately from an external environment model. Dreamer, successor features and trajectory modeling provide precedents, but do not establish that this Rineya adaptation will succeed. [S01, S02, S07]

1.3 Five decisions to freeze before expensive work

1. **Subject:** the exact selected Rineya checkpoint, tokenizer, loader and executor versions; no substitution of a stock Qwen model or the older 344M student.
2. **Authority:** real environment feedback or an independent verifier defines correctness; predicted utility never grades its own success.

3. **Comparison:** the scalar evaluator receives the same context, action sequence and compute allowance as the consequence evaluator.
4. **Information:** predictions never receive future observations, actual future rewards or future actions chosen using hidden observations, unless those actions were explicitly fixed as an intervention.
5. **Promotion:** practical-effect, uncertainty, cost and retention gates are written before sealed evaluation.

The first three reading priorities are successor features for the factorization, DiSPO for distributions of achievable futures, and SF-GEN for the direct language-generation precedent. Trajectory Transformer and masked trajectory models establish that token-like trajectory representations and flexible inference masks are already viable design patterns. The Rineya contribution must therefore be a demonstrated transfer, composition, efficiency or preservation improvement—not a claim to have invented reward tokens. [S02, S04, S05, S07, S09]

2 Scope, provenance and evidence discipline

2.1 Rineya starting point

The September 30 research handbook identifies a historical experimental subject named `Rineya-Titan-Experimental-1.0.0-devfreeze`, with 596,049,920 base parameters. It also records that a laptop route passed short native mechanics checks but encountered a prefix-versus-bulk likelihood discrepancy during capture qualification. Those are historical report statements, not a fresh audit of the current repository. Stage G0 must re-establish the active checkpoint and current qualification state before collecting hidden representations. [I01]

The target deployment envelope is the user's RTX 4070 Laptop GPU with 8 GB of VRAM and constrained host resources. This report uses that envelope for design estimates; actual free memory, operating-system overhead, thermal behavior and current RAM configuration must be measured locally. Do not convert a short-sequence load success into a claim that long-context training fits.

Preserve existing Rineya acquisition and retention work as independent instruments. A base model can remain byte-identical while an attached planner degrades deployed behavior. Consequently, report both **weight preservation** and **system-level behavioral preservation**. Existing composable-skill reports specifically motivate paired evaluation, sealed splits and full resource accounting. [I02]

2.2 Evidence labels

Literature finding means a claim supported by a named primary publication in its own setting. **Historical internal record** means a statement from a previous Rineya report, not independently reproduced here. **Derived example** means transparent mathematics or a constructed example. **Proposal** means a design, threshold or hypothesis requiring execution. **Reference mechanics** means code actually run in the supplied synthetic CPU scaffold. **Native result** is reserved for a measured experiment using the pinned Rineya checkpoint; none is claimed in this delivery.

The annotated register lists 43 primary papers, author project pages and official documentation references. Selected methods were examined for Dreamer, successor features and DiSPO; many surrounding papers were reviewed at the primary abstract or project-summary level. This is a targeted synthesis, not an exhaustive systematic review or a full audit of every cited codebase. Publication venue and version are separated from access date. Recent preprints are explicitly not treated as independent replication.

2.3 Research authorization boundaries

This bundle authorizes no background job by itself. A later implementation handoff should state the repository, parent revision, permitted directories, model identity, data scope, compute budget and stopping rule. The proposed branch is `research/rineya-consequence-world-model-v1`; it has not been created by this deliverable. Synthetic tests may run without a checkpoint. Native extraction, training, model downloads,

paid APIs, cloud rental and remote pushes need the user's applicable authorization and the repository's current policies.

The main scientific question should remain stable even if a particular implementation fails. Preserve reports and failing tests instead of deleting an experiment because it did not support the hypothesis. Conversely, architectural elegance is not a reason to continue spending compute after a well-controlled simpler baseline dominates.

3 Formal model: actions, consequences and goals

3.1 Define the state that the model actually has

Let o_t be an observation and $H_t = (o_0, a_0, o_1, \dots, o_t)$ the available interaction history. The model receives a belief representation $b_t = E_\eta(H_t, e)$, where e contains environment rules and available tools. This is not necessarily the true Markov state. In partially observed tasks, two identical current observations can imply different consequences because their histories differ.

An action is not merely the string `VERIFY`. Define

$$a_t = (\text{operator}, \text{arguments}, \text{budget}, \text{executor version}).$$

The transition record includes the resulting observation, executed status, duration, resource use and independent checks. Let $\phi_t = \Phi(\tau_t)$ encode the consequences of transition $\tau_t = (H_t, a_t, o_{t+1})$. A useful starting decomposition is

$$\phi_t = [\phi_t^{\text{measured}}; z_t^{\text{learned}}].$$

Measured components anchor the model to checkable events. Learned components preserve the user's proposed latent semantic space. Neither is assumed to be intrinsically good or bad. A failed action, information acquisition, elapsed computation and final task success are different events and should remain distinguishable.

3.2 Three related prediction targets

A one-step model predicts $p_\theta(\phi_t, b_{t+1}, d_t \mid H_t, a_t, e)$, where d_t describes termination. A plan-conditioned model predicts consequences for a **specified** candidate action plan A:

$$p_\theta(\phi_{t:t+K-1}, b_{t+1:t+K}, d_{t:t+K-1} \mid H_t, A, e).$$

A successor-feature model predicts a discounted accumulation under an explicitly named continuation policy π:

$$\psi_K^\pi(b_t, a_t) = \mathbb{E}_\pi \left[\sum_{k=0}^{K-1} \gamma^k \phi_{t+k} \mid b_t, a_t \right].$$

These targets are not interchangeable. The same first action can be excellent under a competent continuation policy and poor under one that repeatedly makes mistakes. Every successor target must therefore carry a policy identity or the actual open-loop plan. A target mixed across behavior policies without that information is a mixture, not an optimal-action value.

For variable-duration actions, either use explicit per-decision discounting or use elapsed-time discounting with cumulative duration. State which convention is being estimated. A long tool call must not quietly receive the same latency cost as a cheap token action. The reference scaffold implements finite-horizon discounted features, not an infinite-horizon policy-learning algorithm.

3.3 Utility is a readout, not the world itself

A tractable first reward family is

$$r_g(\tau_t) = w_g^\top \phi_t, \qquad Q_g^\pi(b,a) = w_g^\top \psi^\pi(b,a).$$

This factorization is grounded in successor-feature research. Its advantage is limited but useful: a new weighting of already represented consequences can change values without relearning dynamics. It does not mean that every possible new objective lies in the learned feature span, or that a policy optimized for an old goal remains optimal for a new one. Generalized policy improvement uses a set of supported policies; planning can search other supported candidates. Neither guarantees recovery of unobserved skills. [S02, S03]

A nonlinear utility $u_g(\phi)$ is legitimate, but it changes the requirements. In general,

$$\mathbb{E}[u_g(\phi)] \neq u_g(\mathbb{E}[\phi]).$$

A mean embedding that combines "safe success" and "dangerous failure" is not an actual intermediate outcome. Risk constraints, threshold rewards and conjunctions may require joint samples, richer statistics or a distribution over successor features. DiSPO is a particularly relevant precedent for distributions of achievable long-run features; its dataset-supported reachability is an important limitation to retain. [S04]

3.4 Preference-only goals versus goals that alter behavior

When a goal simply says "value speed more" and the proposed plan is fixed, the predicted physical consequences should not change. Only utility should change. This is a valuable invariance test.

When the goal changes an action's actual execution—for example, a `WRITE_REPORT` operator whose prompt is goal-dependent—the consequences can change. Supply the operative goal to the executor and model. Do not enforce goal invariance where it contradicts the environment. Distinguish **preference context**, **task specification** and **control instruction** in the schema.

3.5 What an embedding can and cannot mean

An embedding is a learned coordinate system. Its dimensions do not automatically correspond to correctness, information or reasoning phases. For an invertible matrix A, setting $\phi' = A\phi$ and $w' = A^{-\top} w$ preserves the scalar reward. Consequently, many geometries can explain the same observations. Nearest-neighbor plots and isolated cosine values are diagnostics, not proof of a unique semantic decomposition.

Meaningful tests concern preserved information and behavior: can a small readout recover held-out observable properties, can new goals be fitted with fewer labels, do controlled action changes cause the predicted consequence changes, and do those predictions improve action selection? A cluster name is assigned after a blinded audit; it is not a training truth because the cluster looks intuitively coherent.

There is also no preference-identification miracle in unsupervised data. The same transition dataset is compatible with w and $-w$. Unsupervised learning can discover structure; something must ground which outcomes are desirable. A verifier, task success flag, preference comparison, explicit goal or chosen intrinsic objective supplies that grounding. DIAYN and preference-learning work illuminate different sides of this distinction. [S17, S20]

4 Relationship to prior art and the defensible contribution

4.1 Four families to distinguish

Sequence-modeling policies. Decision Transformer predicts behavior from history and desired return. It is a useful baseline for goal-conditioned action generation, but a desired return is not an observed future outcome. Generalized Decision Transformer broadens the conditioning statistics; it also highlights why hindsight targets and deployment inputs must be separated. [S06, S18]

Trajectory world models. Trajectory Transformer gives the closest early precedent for representing states, actions and rewards as token sequences. UniMASK and Masked Trajectory Models broaden the tasks expressible by masking. They motivate the interface, not a conclusion that a causal language model should simply be switched to bidirectional attention without requalification. [S07–S09]

Task-relevant latent models. Dreamer and MuZero show different ways of making predictive models useful for decisions. DeepMDP and bisimulation-oriented representation work motivate preserving decision-relevant transition structure rather than reconstructing all raw observations. These approaches also caution against an excessively compressed representation that removes distinctions required by later goals. [S01, S11–S13]

Reusable future-consequence representations. Successor features separate consequences from a reward readout. DiSPO models richer future-feature distributions. SF-GEN already brings successor features to controlled language generation, so “successor features for an LLM” cannot be our novelty claim. [S02–S05]

4.2 The research space that remains worth testing

The proposed contribution is a controlled combination: pretrained compact-language representations; executable, parameterized reasoning or tool actions; learned consequence geometry; goal revaluation; long-distance and cross-subtask dependencies; and preservation-aware integration. A combination is not scientifically important merely because its parts can be connected. It becomes interesting when it provides a reproducible benefit that a matched alternative cannot explain.

Candidate publishable findings include a lower reward-label requirement for new goals; improved choice among plans requiring held-out skill compositions; improved accuracy at equal total reasoning cost; or a learned representation that preserves useful consequence distinctions across domains while scalar evaluators fail. An equally useful negative finding would show that clustering adds interpretability but no control value, or that long-context improvements arise entirely from additional context rather than consequence factorization.

The comparison should not pit a full trajectory Transformer against a scalar MLP with only one local state. A scalar model can use attention, long context, rich hidden features and multiple outputs. Match those capabilities before attributing a gain to the representation. This is an experimental design requirement, not a theorem that the richer model must win.

4.3 Closest precedents: adoption decisions

Reference	Adopt first	Defer or reject as an initial dependency
Successor features	Explicit reward factorization and policy identity	Claims of arbitrary zero-shot optimality
DiSPO	Direct future-feature prediction comparison	Full diffusion outcome generator before a baseline works
SF-GEN	Frozen-LM feature/value comparator	Claims that this is the first language application
Trajectory and masked models	Typed slots and audited masks	Unrestricted future visibility during planning

Reference	Adopt first	Defer or reject as an initial dependency
Dreamer / MBPO	Re-anchoring and model-error gates	Long imagined-RL rollouts at project start
CPC / VICReg / VQ	A staged representation ablation	All objectives and codebooks enabled at once

The current literature register also includes text-world-model and visual predictive-model work through 2026. Those sources refine the roadmap but do not establish a Rineya result. Earlier conversation percentages and venue attributions that were not reverified are not used as evidence in this report. [S36–S39]

5 Architecture and implementation boundaries

5.1 One conceptual model, two implementation routes

The intended architecture is a Transformer that relates history, actions and future consequences in a shared computation. The safest first experiment uses a **sidecar** so representation learning can be debugged without changing Rineya. That is a staging choice, not abandonment of the user's unified-model idea. A later **native typed-slot adapter** implements the same contracts inside the actual Rineya backbone. Compare both rather than assuming that sharing weights is always more efficient.

For the sidecar, a frozen capture adapter extracts a bounded set of Rineya context vectors. A small typed Transformer receives those vectors and candidate actions. It outputs consequence features, next-state summaries and optional uncertainty estimates. The goal readout scores predicted consequences. A controller chooses a feasible action, executes it in the real environment and re-encodes the actual new observation.

For the native route, introduce typed embeddings or projected continuous slots at Rineya's verified hidden width. Train limited adapters and small output heads while retaining the ordinary language route. Use the supported embedding-input interface or a tested wrapper; do not silently resize the language vocabulary and retie its output weights without checking exact language-logit preservation. Stock Qwen3 documentation is an interface reference, not proof that a custom Rineya loader implements it identically. [S42]

5.2 Proposed component contracts

Component	Input	Output / invariant
History encoder	Available text, evidence and events	Time-stamped context slots; no future facts
Action encoder	Operator, arguments, budgets, version	Typed action vectors; executable semantics
Consequence predictor	History and permitted action prefix/plan	Consequence distributions or embeddings
Outcome decoder	Consequence representation	Independent observable targets for grounding
Goal readout	Goal weights or learned task embedding	Utility; no alteration of physical facts
Candidate proposer	Available state and legal operator set	Bounded candidate plans with validity checks
Controller	Predictions, support, constraints	One admitted action or a safe fallback
Recorder	Actual action and actual result	Immutable training/evaluation evidence

The predictor must not receive its targets in its forward signature. In particular, a tensor named `fu-`

`ture_states` should not be accepted by the prospective evaluator. Retrospective representation training may encode outcomes to create targets, but target construction lives in a separate module with a separately audited visibility contract.

5.3 Typed token construction

For event i, form a projected vector

$$x_i = P_{\text{type}(i)} v_i + e_{\text{type}(i)} + e_{\text{time}(i)} + e_{\text{subtask}(i)}.$$

Text uses the ordinary tokenizer and embedding path. Continuous state summaries and consequence slots use learned projections. Operator embeddings distinguish operations; argument encodings distinguish `CALCULATE(2+2)` from `CALCULATE(large_expression)`. Duration and resource encodings prevent the model from treating an expensive verification process as a free semantic label. Type IDs distinguish a proposed outcome from an observed outcome even when their dimensions match.

A candidate serial sequence can be serialized as history slots, an action slot, an outcome-query slot, another action slot and another query. Query slots contain learned placeholders, not ground-truth outcomes. In the first causal version, the query for step t can access only the initial observed history and candidate actions through t. The model learns to integrate the consequences of earlier planned actions implicitly; it does not receive teacher-forced future observations at evaluation.

Outcome vectors do not have to be literal vocabulary tokens. Separate projections and heads can preserve the useful Transformer interface without allocating thousands of arbitrary reward strings. Learned discrete codes are a later representation choice, not a prerequisite for the architecture.

5.4 Pilot tensor specification

The following are proposed screen values, not measured optima. Start with a 128–256-dimensional sidecar, 2–4 layers, 4 attention heads, 32–64 learned consequence dimensions and a small set of grounded features. For native integration, infer width and layer layout from the checkpoint. Do not hard-code stock-model dimensions into persisted captures.

A training batch contains context `[B,Lc,dc]`, proposed actions `[B,H]` plus argument features, target consequences `[B,H,F]`, target-validity masks `[B,H,F]`, step-validity masks `[B,H]`, optional next-observation targets and a split/parent index. Goal weights may have shape `[B,F]` for a measured-feature baseline; learned-latent readouts must fit weights in that latent space using allowed calibration data. A hand-written six-dimensional goal vector cannot simply be dotted into an arbitrary learned 64-dimensional latent.

The predictor should expose both `predict_step` and `predict_plan`. If these share weights, specify their masks. Output a mean and per-target uncertainty only where the training likelihood makes this meaningful. A vector of independent standard deviations is not a full covariance model. Store units and feature-normalization metadata with every model artifact.

5.5 Backbone reuse and target stability

Extract representations at explicit action boundaries. Compare last-token, pooled evidence and multiple selected-layer summaries on development data. Normalization and pooling must be consistent across capture modes. Do not assume a raw subtraction of hidden states from different prompts measures a meaningful semantic transition.

A frozen encoder gives stable targets and cheap caching. A learned encoder can improve task relevance, but then the predictor chases moving targets. Use a separately versioned target encoder, stop-gradient targets and a controlled exponential moving average only after the frozen-target baseline works. Recompute or invalidate caches whenever encoder weights, tokenizer, prompt template, mask, precision or capture location changes. A stale representation cache is a scientific data error, not merely an engineering inconvenience.

The CPU scaffold intentionally uses a tiny synthetic state input rather than pretending to load Rineya. Its learned latent bottleneck is grounded through feature prediction. It demonstrates the interface and tests selected invariants; the fully self-supervised latent-consequence arm and native checkpoint integration remain planned work.

6 Executable actions and consequence semantics

6.1 Give each token an operational definition

`RETRIEVE` must name a retrieval corpus, query generator, top-k, evidence budget and tool version. `VERIFY` must identify what is checked and the verifier's authority. `RECUR` must specify a qualified recurrent module, depth, gating value and termination rule. `ANSWER` must identify the format and final evaluator. Two runs with the same operator name but different prompts or tool versions are different action mechanisms.

An action record contains its ID, parent events, arguments, preconditions, anticipated resource needs and permitted side effects. The executor returns a status from a fixed set such as success, invalid precondition, timeout, tool error or explicit refusal. A failure is not silently dropped from training. Dropping unsuccessful calls biases the predictor toward imaginary reliability.

For the first real Rineya pilot, use only a few controllable operations: retrieve a document span from a fixed corpus, execute a bounded arithmetic expression, check an answer against a symbolic constraint, and submit a final answer. Broader cognitive labels such as "decompose" or "think harder" can follow once their execution contracts are stable. Learned action representations are useful precisely when there is actual structure in the available operations. [S10]

6.2 Grounded feature dictionary

Use a versioned dictionary with units, observability and label source. A initial example includes terminal task success, a newly verified constraint, contradiction detected, useful evidence obtained, elapsed milliseconds, generated tokens, tool failure and irreversible constraint violation. Some are binary, some counts, some nonnegative costs. Predict their distributions or normalized continuous values as appropriate.

Do not equate an observed reduction in the model's entropy with a correct conclusion. Call it "entropy change," not "information gain," unless the experiment provides a well-defined information target. Similarly, "progress" requires an external measure such as satisfied prerequisites or a reduction in known distance to a simulator goal. A fluent narrative that says progress occurred is not a measurement.

Earlier brainstorming placed retention beside compute and correctness in a reward vector. For ordinary inference actions, that is misleading: the frozen model does not change its weights when it retrieves a document. Retention belongs in **cross-update evaluation**, unless the selected action is itself a training update with measured downstream effects. Keep the feature inventory honest about the timescale of each quantity.

6.3 Avoid double-counted reward

A step that verifies a prerequisite can deserve a shaping signal, but the final success reward should not be paid repeatedly after success. The plan objective must specify whether intermediate signals are explanatory features, auxiliary supervision or actual additive rewards. Those roles differ.

For example, store elapsed time as a positive measured quantity and place a negative coefficient in the utility readout. Do not encode negative time and also multiply by a negative penalty. Count candidate-generation and scoring costs in system reporting even though they do not appear in the environment transition. An improved controller that spends ten times as much on evaluation has not demonstrated compute efficiency.

6.4 A constructed example

A task requires a key, an opened door and a collected object. A shortcut collects the object immediately but causes damage. Consequence features are `[object_gained, damage, action_cost]`. A safe three-step route yields a cumulative vector `[1,0,3]`; a one-step shortcut yields `[1,1,1]`.

With weights $w_{\text{safe}} = [10, -20, -1]$, the route scores 7 and the shortcut scores -11. With $w_{\text{fast}} = [10, -1, -2]$, the route scores 4 and the shortcut scores 7. These are constructed arithmetic values, not model predictions or empirical results. The physical futures did not change; the preference did.

This example is intentionally simple. If success also requires no damage, then utility is a conjunction rather than this linear readout. Add the relevant joint feature, predict a joint distribution, or use a nonlinear scorer with an appropriate outcome distribution. Do not use linear scoring outside the feature family and blame the world model for an incorrectly specified objective.

7 Learning the consequence space and testing clustering

7.1 Preserve the discovery hypothesis

The first measured-feature model is a debugging instrument. The actual research hypothesis is that a representation learned from transitions contains useful distinctions we did not fully prescribe. A credible program therefore needs three parallel representation tracks: measured features, learned continuous features, and learned discrete or clustered features. Each must be compared using the same action data and the same downstream reward-label budget.

For learned continuous features, encode observed transition or trajectory outcomes with a target encoder and predict that representation from the preceding history plus action. Add a small decoder for independently checkable properties. This makes it possible to detect a representation that predicts itself perfectly while carrying no useful information. Use a held-out linear probe and a nonlinear probe separately: a high-capacity probe can hide a poorly organized representation.

Predictive contrastive learning, variance/covariance regularization and vector quantization supply alternative mechanisms. They should be introduced one at a time. None guarantees that the resulting latent dimensions correspond to useful rewards. [S14–S16]

7.2 Clustering protocol

Begin with train-only standardization and a continuous baseline. Fit k-means to TRAIN embeddings for a small predeclared set such as $K \in \{16, 64, 128\}$. Freeze centers before development or test assignment. Compare hard assignments, soft distances and the unclustered vector under equal downstream model capacity. A random projection and random-label cluster control help identify whether a gain comes from regularization or parameter count rather than semantic grouping.

For a vector-quantized arm, log code usage, entropy, empty/dead codes, assignment turnover, quantization distortion and downstream regret. Resetting dead codes is a training intervention that must be logged and must not use evaluation examples. A codebook can stabilize a vocabulary of outcomes, but can also merge rare catastrophic failures with frequent harmless events. Evaluate failure distinctions explicitly rather than relying on average reconstruction error.

Inspect clusters only after the quantitative screen. Draw a blinded sample per cluster, have an evaluator describe observable commonalities, then test those descriptions on fresh examples. Do not let a compelling hand-assigned name turn into an unsupported assertion that the model has discovered a cognitive primitive.

7.3 Sparse grounding experiment

Use nested label budgets, for example 1%, 5%, 10%, 25% and 100% of the same TRAIN parent set. All representation arms receive the same unlabeled transitions; all receive identical labeled parents at each bud-

get. Count verifier calls, human comparisons, teacher generation, rejected traces and environment execution separately. “Unlabeled” transitions may still be expensive to collect.

Fit the utility readout using only the permitted calibration labels. Separate two transfer tests. **In-span revaluation** changes weights on already represented features. **New-outcome transfer** introduces a property not explicitly included in prior grounding. The latter can benefit from learned latent structure, but requires evidence that the representation retained that property. Neither test may reuse variants of held-out parents in pretraining or clustering.

The principal label-efficiency endpoint is the number of grounding labels required to reach a preregistered action-regret or success target. Report uncertainty on that curve and the total data-production cost. A claim that clustering uses “10% of labels” is not justified by selecting one favorable random label subset.

7.4 Representation failure diagnostics

Track feature variance, covariance spectrum, nearest-neighbor stability, decoder quality, cross-goal sufficiency, action sensitivity and nuisance sensitivity. Compare prediction under real actions against shuffled action labels. If shuffling barely matters, the model may predict outcome frequencies from the state while ignoring the proposed intervention.

Do not force all successful outcomes to be neighbors. Different successful routes can have different costs, risks and prerequisite effects. A contrastive objective that collapses them may improve a binary success probe while harming goal transfer. Likewise, “different action” is not automatically a negative pair: two tools may cause the same observable effect.

8 Information flow, masking and counterfactual validity

8.1 Three masks, three meanings

Prospective causal transition mask. A query for the consequence of action a_t may see the observed prefix and candidate actions through t. It may not see later actions, outcomes or rewards. Changing a_{t+1} must not change the predicted consequence at t. This is the default dynamics interface.

Prospective whole-plan mask. A plan-value or plan-outcome query may see a fully specified open-loop candidate plan, including later proposed actions. All unobserved future outcomes remain absent or masked. This can score the total plan, but it should not claim that later choices physically cause earlier consequences. A suffix-aware auxiliary scorer and a causal dynamics head may share some parameters while maintaining separate visibility tests.

Retrospective mask. A representation learner may encode a completed trajectory to learn outcome structure or redistribute credit. Its output is a training target or retrospective diagnostic, not a prospectively available state. Confusing this mode with the deployed evaluator is a particularly easy way to manufacture excellent validation scores. Masked trajectory research motivates these interfaces; deployment validity is our responsibility. [S08, S09, S18]

8.2 The subtle future-action leak

Even when rewards and observations are masked, a recorded future action can reveal a hidden observation. A logged policy may choose `GO_LEFT` only after seeing a secret clue. Showing that future action to an earlier predictor leaks the clue. An action column is not safe merely because it is called an action.

For the first plan-conditioned dataset, choose candidate plans **before** executing their branches, using only the shared starting observation and allowed information. Record `plan_origin=open_loop_precommitted` or a policy-tree contract with explicit observable branch conditions. For retrospective trajectories selected by an adaptive policy, use causal prefixes or include an honest policy-conditioned target; do not treat the realized suffix as a freely chosen plan.

8.3 Predictive association versus causal effect

Logged trajectories can be confounded by environment difficulty, hidden state and behavior-policy competence. To learn an action effect, prefer exact simulator snapshots and branching interventions: restore the same starting state, execute different permitted actions, and compare actual consequences. Keep all branches in the same data split. In stochastic tasks, log random seeds and use repeated draws; shared random numbers can reduce variance but do not turn stochastic outcomes into deterministic facts.

A language-model hidden-state intervention is not automatically an intervention on an external task variable. First test whether its predicted changes correspond to observable environment changes. Only then study stronger internal causal hypotheses. Attention maps, cluster assignments and decoded labels provide hypotheses for interventions, not proof of causal structure.

8.4 Tests that must fail before the fix and pass after it

Perturb a hidden future reward and confirm the prospective input bytes and outputs are unchanged. Perturb future action suffixes and confirm causal earlier outputs are unchanged. Introduce a future timestamp in a supposedly known observation and reject the record. Replace a legal precommitted plan with a behavior-selected suffix and require a different mode or an error. Change padding, batch order and unrelated examples and verify the expected numerical tolerance.

Mask Boolean conventions deserve explicit tests: the `torch.nn.Transformer` API marks blocked positions with `True`; other attention APIs can use the opposite convention. A shape-correct mask can therefore be completely wrong. Pin the installed API and test information flow behavior, not just matrix dimensions. [S41]

8.5 Parallel output is not joint uncertainty

A Transformer can produce reward or feature predictions at every query position in one pass. Independent heads at those positions generally estimate marginals. They do not guarantee coherent joint samples across steps, tasks or reward dimensions. If a task requires “both subtasks succeed” or a correlated failure mode, test the joint event directly. Later options include a shared latent scenario variable, an autoregressive consequence decoder or a distributional trajectory model. Count their sampling cost in comparisons.

9 Data program and environment ladder

9.1 Start where errors can be localized

The first environment should be simple enough that an oracle can exhaustively evaluate candidate plans. The supplied KeyDoor simulator is a mechanics example, not the final benchmark. Build a broader symbolic suite with prerequisite chains, bounded program execution, document-evidence retrieval and resource-constrained parallel subtasks. Each environment should expose immutable snapshots, legal-action validation, independent outcome checking and a controllable observation function.

Use separate axes for context length, decision horizon, branching factor and partial observability. A 16,000-token prompt with one action is not a long-horizon task. A 20-action sequence with a tiny state is not a long-context reading task. A fair evaluation varies one axis at a time, then tests their combinations.

Only after the exact simulator gates should the program move to TextWorld or ALFWorld-style interactions, and later to a sandboxed web benchmark. These environments offer richer interactive structure but introduce additional language and state-aliasing failure modes. External benchmark acquisition, licenses and versions remain a preflight task. No real purchases or account changes belong in the training environment. [S31–S33]

9.2 Collection policy and coverage

Collect a mixture of legal random exploration, heuristic actions, the frozen Rineya proposal policy and deliberately selected alternatives. Log which policy selected each action and its available information. Retain failures, invalid attempts, no-ops and timeouts with explicit status labels. An all-successful dataset teaches the model little about the actions it most needs to reject.

For a practical pilot, propose 2,000 independent training parents, 300 development parents and 500 sealed evaluation parents, with several matched branches per parent. These are **starting budget proposals**, not a power calculation or a guarantee of sufficient coverage. Use the development pilot to estimate variance and effect sizes; freeze the final sample size before opening the sealed set. If the required sample size exceeds the compute budget, narrow the claim or improve measurement rather than reporting an underpowered absence of harm.

Balance training by informative transition types, not only by trajectory count. A million repeated no-ops can dominate reconstruction loss. Preserve natural prevalence for a separate calibration set so a balanced training sampler does not produce misleading real-use probabilities. Report support for each operator, argument type, context band, goal family and failure mode.

9.3 Data contract

Every trajectory record needs a schema version, parent ID, counterfactual family ID, episode ID, split, environment/version hash, initial observation, source history and observation timestamps. Events carry action/operator IDs, arguments, executor hash, preconditions, start and end times, dependencies, actual outputs, statuses, feature values, feature-validity masks, reward-source IDs, termination and truncation. Optional hidden-state references carry model/tokenizer/capture hashes and byte offsets.

Keep **observed facts**, **proposed actions**, **predicted futures** and **retrospective annotations** in distinct fields. Do not overload an `outcome` field with whichever version is currently convenient. A typed schema reduces the chance that a training target later becomes an inference feature.

Prediction logs need candidate-set identity, all candidate scores, uncertainty/support estimates, chosen action, fallback reason, actual outcome and resource totals. Storing only the selected action hides whether gains come from the proposer or scorer. Store all candidates before execution so later analysis cannot silently change the comparison set.

9.4 Split integrity

Partition by a stable problem-family key before generating branches, paraphrases, goal variants or labels. All descendants remain together. When tasks share a generator template or source document, use a sufficiently broad grouping key to prevent template leakage. A random row split of trajectories from the same parent is not a generalization test.

Normalizers, codebooks, representation pretraining, reward readouts and hyperparameter selection must respect the split boundary. Separate a **new-goal calibration set** from the held-out evaluation parents. New goal labels may be permitted during adaptation, but labels from evaluation scenarios are not. Record this distinction in the experiment contract rather than calling every goal adaptation “zero-shot.”

Preserve a frozen ledger of hashes for raw data, parent partitions and generated variants. Deduplicate before split finalization and recheck after generation. Where a benchmark has its own train/test partition, retain it and add internal development splits only within training data.

9.5 Label provenance and privacy

Each feature label records how it was obtained: simulator state, executable checker, deterministic tool, human comparison or learned proxy. Report accuracy against independent checks for any learned labeler. Use separate model identities for generator and evaluator when appropriate, and disclose shared training or correlated errors. Never label the reward model’s own prediction as ground truth.

Do not include private email, real credentials, personal identifiers or proprietary documents in synthetic training data by default. Tool outputs are untrusted data, not instructions to alter the experiment. Redact secrets before logging; retain enough provenance to reproduce a sanitized example. Store only the hidden representations needed for the experiment, with explicit access and retention policies.

10 Training recipe and staged objectives

10.1 Stage A: measured consequences as an instrument

Train a small model to predict known consequence features from initial state/history plus a precommitted action prefix. Use binary cross-entropy for Bernoulli events, cross-entropy for categorical state variables and a suitable regression likelihood for measured continuous quantities. The CPU scaffold uses mean-squared error for simple synthetic numerical features; production binary probabilities require the corresponding calibrated heads.

Normalize continuous targets using TRAIN statistics. Preserve physical units in the artifact. Apply feature-validity masks so an unavailable label is not treated as zero. Start with a fixed-length short horizon and a single frozen encoder. Require the model to overfit a tiny diagnostic set before larger experiments; failure there usually indicates a contract, optimization or observability problem.

10.2 Stage B: learned consequence representations

Let $z_t = E_{\bar{\eta}}(\tau_t)$ be a target outcome representation and $\hat{z}_t = F_\theta(H_t, a_t)$ its prediction. A basic predictive loss is

$$\mathcal{L}_{\text{pred}} = \frac{1}{|M|} \sum_{t \in M} \|\hat{z}_t - \text{sg}(z_t)\|_2^2.$$

The stop-gradient operator prevents the predictor from trivially moving both sides through that term, but does not alone prevent the whole representation from collapsing. Combine it with a grounded decoder, contrastive prediction or an explicitly tested variance/covariance objective. Start with one anti-collapse mechanism, not all of them. CPC and VICReg provide relevant objective families; our exact transition augmentations and feature targets remain hypotheses. [S14, S16]

An auxiliary observable loss can decode state predicates, action status and elapsed resources. Evaluate both the auxiliary prediction and downstream utility. A representation might predict all easy surface properties while dropping a rare critical prerequisite. Stratified diagnostics and held-out conjunctions are needed to expose that failure.

10.3 Stage C: sparse utility grounding

Freeze or slowly adapt the consequence encoder and fit $u_\omega(z, g)$ using verified returns or preference pairs. Use a linear readout first because it exposes what is actually reusable. Compare a small nonlinear readout only after the linear result is known. For pairwise labels, train a calibrated preference model rather than pretending that every comparison reveals an absolute numeric reward.

When fitting goal weights, constrain them to the declared feature family or record the extrapolation. New-language goal prompts must be translated to a versioned task representation and tested against explicit objective definitions. Do not claim immediate generalization from a natural-language goal embedding without evaluating that mapping.

10.4 Stage D: future features and consistency

Add direct predictions of discounted future consequences at horizons 1, 2, 4 and 8. For a policy-conditioned successor head, enforce the correct continuation-policy identity. A finite-horizon consistency target has the

form

$$\psi_K^\pi(b_t, a_t) \approx \phi_t + \gamma\, \mathbb{E}_{a' \sim \pi}[\psi_{K-1}^\pi(b_{t+1}, a')].$$

Bootstrap only where continuation semantics are valid. True terminal states have no future contribution. A time-limit truncation is not automatically a terminal state; use an explicit modeling convention. The environment API distinguishes these cases, but the learner must still handle them correctly. [S43]

Compare direct successor prediction with recursive one-step rollouts. The former can reduce accumulated prediction steps but hides intermediate structure; the latter supports detailed replanning but can accumulate error. A hybrid is a research option, not the default until these failure modes are measured. [S04, S22]

10.5 Objective budget and optimization

A later combined objective may contain predictive, observable, reward, variance, contrastive, codebook and retention terms. Record each coefficient, its units, its gradient contribution and its activation stage. Do not present a single long loss equation as an implementation recipe. Begin with observable prediction, then add one mechanism at a time.

For the sidecar, propose AdamW with a small development-only sweep over learning rates such as 10^{-4}, 3×10^{-4} and 10^{-3}, fixed token/transition exposure, early stopping on an independent development criterion, gradient clipping and finite-loss checks. These are engineering starting points, not paper-derived optimal settings. Log samples seen, optimizer steps and total modeled tokens. Match baselines on training examples and total tuning budget, not only epochs.

For native adapter training, verify forward/gradient parity on a small case, then train only registered parameter sets. LoRA and quantized-base adaptation are reasonable mechanisms, but their efficiency does not certify retention or numerical fidelity. Quantization must be evaluated as a separate variable. [S27, S28]

10.6 Representation drift and replay

When the representation encoder changes, old goal weights and codebook centers can become invalid. Either keep the encoder frozen, version and refit dependent readouts using allowed data, or train an explicit alignment map and evaluate its error. Store representation version in every prediction record. A resumed training run that silently combines old caches with new encoder weights is invalid.

Retain a small, fixed rehearsal set for language behavior and previous task capabilities only when native weights are adapted. Do not use sealed retention tests as rehearsal. Record update order and test acquisition/retention separately so a model cannot pass preservation merely by learning nothing new.

11 Planning, uncertainty and real-environment feedback

11.1 Separate proposing from evaluating

Generate a bounded set of legal action plans using fixed heuristics or the frozen Rineya proposer. Validate syntax, preconditions, dependency constraints and permissions before scoring. Cache the candidate set and its hash. Every evaluator in a comparison receives exactly that set.

For an admitted candidate A, score an expected task utility and a development-calibrated model-risk penalty:

$$J(A, g) = \mathbb{E}[U_g(\widehat{\Phi}(A))] - \beta\, \widehat{\mathcal{U}}(A).$$

This is a heuristic conservative score unless a formal bound is separately established. Enforce hard constraints through a feasible-set filter, not through arbitrarily large learned penalties. A permission violation must not become acceptable because the model predicts enough reward elsewhere.

Execute only the first admitted action, observe the actual result, update the history and replan. An open-loop plan is a proposal, not a commitment to ignore new evidence. A safe fallback can be a verified tool, an inexpensive direct answer on a qualified class, a request for missing information, or abstention under the task's rules.

11.2 What uncertainty means

Separate stochastic outcome variability from lack of model knowledge. In a deterministic toy environment, residual uncertainty primarily reflects model error or missing state; in a stochastic environment, repeated execution can vary even under a perfect model. Ensembles are a practical baseline, not an oracle for ignorance. Test whether uncertainty predicts error under held-out actions, longer horizons and shifted contexts. [S24, S25]

Report calibration by operator and difficulty as well as overall. Use Brier score or log loss for probabilities; reliability plots, empirical coverage and interval sharpness for continuous outcomes. Define the abstention threshold on development data and freeze it. Measure coverage and real task quality together: rejecting everything is not a useful uncertainty system.

Candidate search itself can exploit the evaluator. Sweep plan count and horizon, and compare predicted gains with actual utility. If predicted reward increases while real utility falls, the controller is exploiting model error. Keep failed cases as a dedicated stress set rather than retraining on sealed evaluation failures. Proxy-reward overoptimization and offline distribution shift make this an essential test. [S23, S26]

11.3 Direct selection, distillation and RL

The first controller can choose the highest-scoring permitted action without online policy-gradient training. That does not make it fundamentally unrelated to RL: it is a learned decision system trained from interaction data. The question is which learning components are needed, not which label is more impressive.

A later distilled policy can imitate the choices of a qualified, more expensive planner, reducing inference cost. Verify that the teacher's preference remains valid under real outcomes. Imagined RL is later still: mix short simulated trajectories with real anchors, use conservative support checks and validate every promoted policy in the actual environment. The Dreamer and MBPO inspirations support this staged idea, not a reason to start with unrestricted self-generated reward loops. [S01, S22]

11.4 Evaluation of control versus representation

Keep three levels of results separate: prediction fidelity, decision quality within a fixed candidate set, and end-to-end agent performance including proposal failures. A strong scorer cannot select an action that was never proposed. Report the candidate oracle gap to show whether further improvement should target the proposer or evaluator.

A model can predict many irrelevant details accurately yet rank actions poorly. Conversely, it can rank familiar candidates well without generalizing dynamics. Both are legitimate limited results. The "world model" label should be scoped to the evaluated environment, observable variables and intervention coverage. Recent language-world-model work similarly motivates evaluating fidelity and actual agent utility separately. [S38]

12 Long context, delayed credit and parallel task structure

12.1 Long-context evaluation grid

Propose context bands of 512, 2,048, 4,096 and 8,192 input tokens, subject to measured hardware limits; action horizons of 1, 2, 4 and 8; and dependency distances varied independently. Hold total relevant information fixed while adding distractors in one test. In another, increase the number of genuinely relevant prerequisites. A context-size configuration is not a claim that the model uses that context effectively.

Use exact synthetic references for early tests: a constraint introduced near the beginning changes which final action is legal, two distant facts must be combined, or an intermediate tool response invalidates an earlier assumption. Report position sensitivity and truncation behavior. LongBench and RULER offer later external context-evaluation references, but do not replace interactive tests. [S29, S30]

For long raw prompts, compare full-history capture, bounded evidence retrieval and hierarchical event summaries. Count encoding and retrieval costs. A compressed history can make the sidecar cheap while discarding the only fact that distinguishes two actions. Test that possibility using adversarially placed prerequisites rather than trusting a summarizer's own explanation.

12.2 Delayed credit without hindsight leakage

A useful early action may have no immediate success reward. Supervise the eventual consequences while preserving when evidence becomes observable. A retrospective return-decomposition model can propose credit assignments; a prospective decision model must predict those outcomes from information available before acting. RUDDER is relevant to delayed credit, but an attention link from an early action to a final reward is not automatically causal attribution. [S19]

A useful ablation compares terminal-only grounding, measured prerequisite features and learned future-consequence embeddings. Match the final objective and label costs. Intermediate supervision may improve training, but then the gain is not purely architectural; the study should identify the contribution of extra feedback.

12.3 Represent parallel tasks as events and dependencies

A parallel task is not simply a serial action list with subtask tags. Define a directed acyclic graph of prerequisite relationships. Each event has a start time, completion time, dependencies, resource claims and output-availability time. Two actions may be independent in logic but compete for the same GPU, file lock or API quota. A plan is feasible only when both dependency and resource constraints hold.

Begin with synchronous rounds: independent actions launch together, then their outcomes become available together. Later add asynchronous completion, failures and cancellation. A query at time t may attend only to outcomes available by t, not to a sibling task that finishes later. Proposed action metadata can be visible according to the selected planning mode; actual results cannot arrive early because their token was serialized earlier.

Test concurrency benefits against a serial controller at equal resources and against an oracle scheduler. Metrics include task success, makespan, resource cost, constraint violations, wasted work and cross-subtask error propagation. A shorter action sequence is not necessarily a shorter wall-clock run.

12.4 Cross-task consequence interactions

When two tasks must both succeed, marginal success probabilities are insufficient unless independence is justified. A shared failure source can make the joint success probability very different from the product of marginals. Include joint outcome labels, shared scenario variables or sampled joint trajectories for this stage.

Test whether the model learns reusable prerequisites rather than task IDs. Hold out new compositions of familiar operations, permutations of independent subtasks and changes in resource contention. Preserve counterfactual families in a single split. A system that succeeds only on the serialized order seen in training

has not learned the claimed parallel structure.

12.5 What attention does not buy for free

Attention provides a mechanism to connect positions and train on many positions in parallel. It does not enumerate an exponential plan space, make all future observations available, infer causal structure from any dataset, or execute tools simultaneously. The plan must still have a proposer, a feasibility check and a measured execution budget. The project's hypothesis is that shared contextual computation improves prediction and ranking—not that attention abolishes planning. [S40]

13 Experimental program and promotion gates

13.1 Hypotheses that can be falsified

H1, useful representation: learned consequence features improve independently measured candidate selection over a matched scalar Transformer. Falsify the claimed representation advantage when the effect disappears after matching context, parameters, labels or tuning budget.

H2, transferable utility: a frozen consequence representation supports new in-span goals with fewer grounding labels than fitting a new scalar evaluator. Falsify the transfer claim when it depends on evaluation-parent leakage, a task-specific encoder update or reward properties not actually represented.

H3, temporal composition: trajectory modeling helps when useful outcomes depend on multiple separated actions. Falsify the long-range mechanism when gains remain unchanged under removal of dependencies, or are explained by a larger context window alone.

H4, discovered structure: clustering or learned discrete codes improve label efficiency, compositional transfer or interpretability without harming rare outcomes. Falsify the benefit when continuous or random-projection controls perform equally well, code usage collapses, or clusters reflect surface task IDs rather than consequences.

H5, deployable integration: the native typed-slot route or a sidecar improves actual task performance within the target cost and preservation envelope. Falsify deployment readiness when the gain vanishes after counting planning overhead, fails under the real executor, or harms protected behavior beyond the preregistered margin.

13.2 Minimal model matrix

ID	Model / representation	Question isolated
B0	Legal random or simple heuristic choice	Environment difficulty and trivial strategies
B1	Frozen Rineya direct action/answer	Existing deployed baseline
B2	Scalar trajectory Transformer	Strong non-factorized evaluator
B3	Measured consequence features + goal readout	Benefit of explicit factorization
B4	Learned continuous consequences + readout	Added value of learned geometry
B5	B4 with train-only clustering or VQ	Added value of discrete structure
B6	Policy-conditioned successor features	Direct future accumulation versus rollout
B7	Native Rineya typed-slot adapter	Backbone-sharing benefit and retention cost
O1	Exact simulator evaluator on same candidates	Candidate-set ceiling; not deployable baseline

Do not run all combinations immediately. B0, B2, B3 and O1 validate the environment and contracts. B1 enters when native capture and action execution are qualified. B4 is the central scientific arm. B5, B6 and B7 are conditional extensions. A scalar baseline must have a comparable representation-learning budget; otherwise an apparent factorization gain may come from additional pretraining.

13.3 Staged gates

G0: provenance and mechanics. Pass schema, split, snapshot, mask, finite-loss and deterministic replay checks. Pin source identity and demonstrate capture agreement within a declared tolerance. Required output: a qualification report with no native-quality claim.

G1: environment and instrument. The oracle solves the intended class; a tiny diagnostic set can be fitted; a measured-feature model learns nontrivial action dependence. Required output: failures stratified by observation ambiguity, label validity and optimization. A broken instrument blocks later architecture comparisons.

G2: learned-representation utility. Compare B2/B3/B4 on paired held-out parents. Primary endpoint: actual utility or task success under a fixed candidate set. Secondary endpoints: consequence fidelity, label efficiency and runtime. Required output: effect estimate, uncertainty interval and resource accounting.

G3: reuse and discovery. Run nested grounding budgets and held-out goals. Add clustering only after the continuous result. Required output: learning curves with identical labeled subsets and a rare-failure audit. A pretty latent plot is insufficient.

G4: composition and horizon. Increase dependency distance, horizon and DAG complexity separately. Required output: free-running prediction and actual-control results, including model-exploitation and support stress tests.

G5: native adaptation and preservation. Add typed slots/adapters behind a mode switch. Required output: checkpoint invariance where promised, acquired world-model skill, old-behavior noninferiority, measured laptop memory and system latency.

G6: optional policy improvement. Distill a qualified planner or add short imagined RL. Required output: real-environment improvement beyond the non-RL controller at matched data and compute. Simulation-only gains are not promotion evidence.

13.4 Sample size and statistical unit

The primary independent unit is the **parent problem or environment family**, not every correlated branch, goal relabel or token. Aggregate paired outcomes within parents, then use a parent-level paired bootstrap or an appropriate clustered model. Preserve gained/lost cases for binary results. Training-seed variability is a separate uncertainty source; report seed-level results and a hierarchical summary where justified. The statistical-precedence literature motivates avoiding confident aggregate claims from fragile run summaries. [S34]

Use a pilot to estimate variance and choose a final sample size before sealed evaluation. As an illustrative planning calculation, if paired parent-level utility differences have standard deviation σ, a rough normal approximation for detecting a mean difference δ at 5% two-sided significance and 80% power is

$$n \approx \left(\frac{(1.96 + 0.84)\sigma}{\delta} \right)^2 .$$

This is a derived approximation, not a substitute for clustered, binary or heavy-tailed simulation-based power analysis. With $\sigma = 0.25$ and $\delta = 0.03$, it suggests roughly 545 independent parents. More correlated branches do not create 545 independent parents.

Predeclare one primary comparison and a small set of secondary questions. Correct or clearly label exploratory multiplicity. Freeze model selection on development data. Sealed evaluation is not a hyperparameter search tool.

13.5 Practical effect, preservation and cost

A suggested contract requires a positive primary paired confidence interval, a preregistered minimum meaningful effect, and no unacceptable cost or retention violation. The actual effect threshold must be chosen using the task scale and pilot variance. Configurations deliberately leave final promotion thresholds unset until that decision; a null threshold means **blocked**, not "no threshold required."

For preservation, select a noninferiority margin on protected capabilities. Failure to reject a difference is not evidence of no harm. Require the lower bound on new-minus-base performance to exceed the allowed loss margin, while separately showing acquisition. A frozen base weight hash is necessary for a strict frozen route but does not certify behavior with the new controller. [I02]

Report peak allocated and reserved GPU memory, process-tree RAM, disk bytes, model and adapter parameters, encoding time, candidate generation, scoring, tool execution, total latency and failure retries. Compare quality at equal wall-clock or token budgets as well as raw accuracy. Never hide a larger candidate search behind "same model size."

13.6 Decision outcomes

Promote a mechanism when it clears the declared accuracy, uncertainty, preservation and cost gates. Refine it when the failure is a diagnosed contract or coverage issue. Pivot to the simpler baseline when the extra mechanism provides no useful benefit at the agreed budget. Preserve a dormant extension when prerequisites are unmet; do not describe it as empirically disproved when it was never qualified.

14 Resource engineering for an 8 GB laptop

14.1 Memory arithmetic before kernel tuning

For the historical 596,049,920-parameter subject, BF16 weights alone require approximately 1.11 GiB. A rough 16-bytes-per-parameter full-training accounting gives approximately 8.88 GiB before activations, attention workspaces and runtime overhead. These are derived storage estimates, not measured device usage. They explain why full fine-tuning is not the first route in an 8 GB envelope.

A small sidecar can be trained from cached frozen representations with no Rineya weights resident during that training phase. Native adapters reduce optimizer state but still require activations for gradient propagation. Quantization can reduce frozen-weight memory, but does not eliminate context activations, and the exact quantized route must pass capture and behavior parity checks. [S27, S28]

Caching strategy matters. A single BF16 vector of width 1,024 for each of 100,000 events occupies about 195 MiB. Saving all 28 such layer vectors would use about 5.34 GiB before metadata. Saving every token of every context is much larger. These are illustrative dimensions; the actual checkpoint configuration remains authoritative. Cache selected action-boundary summaries first, and use chunked files rather than loading the entire corpus into host memory.

14.2 Three operating envelopes

CPU mechanics: no Rineya checkpoint, tiny synthetic states, short horizons and small Transformer. This is the supplied reference route. It validates accounting and interfaces, not language intelligence.

Frozen capture plus sidecar: load the qualified Rineya route only to extract bounded representations; flush cache shards; train the small predictor separately. Begin at 512 tokens and one candidate microbatch. Increase context or candidate count only after measuring peak memory and timing.

Native adapter experiment: register trainable modules, enable appropriate checkpointing, use a microbatch of one if necessary, accumulate gradients and remeasure the exact precision/cache/mask combination. Increase one dimension at a time. A previous short forward pass does not authorize an 8,192-token backward pass.

Reserve explicit VRAM and host-memory headroom, and stop before the operating system enters uncontrolled paging. The suggested native configuration sets a conservative peak-allocation admission target rather than assuming the whole advertised 8 GB is available. Reserved-memory and process-tree-RAM checks remain necessary. Guard against thermal throttling when comparing runs separated in time.

14.3 Numerical qualification

Before capture, compare single-prefix and bulk causal predictions at the same token positions. Check padding side, position IDs, attention masks, cache use, dropout mode, precision and fused-kernel settings. Compare both selected hidden vectors and raw next-token log probabilities where accessible. A small score discrepancy can change candidate ranking near a decision boundary.

Test save/resume continuity, immutable base hashes, optimizer-step counts and accumulation scaling. Ensure that a skipped or failed batch does not silently count as an update. Store maximum errors and the declared tolerances, not only a pass/fail word. The historical Rineya capture issue makes this gate especially important, but the report does not assume that the historical issue remains unresolved today. [I01]

15 Preservation and integration with existing Rineya work

15.1 Keep the base anchor and the scientific question separate

The consequence model is a shared interface that can serve several existing directions. It does not replace their controls. Hidden-state research supplies candidate representations; TAPS/CDE supplies possible compute-allocation actions; CSS and retention work supply adapter and preservation mechanisms; post-training supplies trajectories and optional policy updates. Integrate through contracts and shared logging rather than immediately merging every experimental branch.

Adaptive recurrence is a later action family because it requires a reliably executable recurrence mechanism and a measured benefit. A world model cannot rescue a recurrence operator that does not produce useful state changes. First compare the actual effects of additional recurrence against ordinary extra tokens or a matched tool action. Then ask whether the consequence predictor can choose when to use it.

For skill composition, keep acquisition, old-skill retention and joint execution separate. A world-model adapter may retain old answers but fail when two skills must interact in one plan. Evaluate held-out compositions and real action sequences, not only separate skill test scores. Low cosine similarity between parameter deltas does not certify functional independence. [I02]

15.2 Native mode isolation

A `language` mode should preserve the original prompt path, tokenizer, model weights and output head. A `consequence` mode adds the registered projections/adapters and heads. Unit-test that disabling the new mode restores the original forward behavior within the required tolerance. If a shared adapter changes normal language behavior, report it as a changed model, not an unchanged base because only a small parameter fraction was trained.

Do not merge an adapter or delta into the anchor merely to simplify packaging during research. Persist the anchor identity, adapter identity, target modules, precision and merge recipe separately. Quantized or low-precision deployment can alter small updates; qualify the artifact that will actually run, not only a higher-precision training copy.

15.3 Teacher and data reuse

Existing post-training traces can be useful only when their action boundaries, executor semantics and reward provenance are known. Free-form reasoning text is not automatically an executable trajectory. Tag unsupported retrospective labels as such. When teacher-generated actions or labels are used, count teacher cost and retain the possibility that multiple models share the same systematic error.

A distilled planner should be judged against real outcomes, not merely its ability to reproduce a teacher's preferred plan. Retention replay must come from allowed training data. Old sealed suites remain evaluation-only, and current experiment failures must not be inserted into training while preserving the same claimed test split.

16 Engineering, safety and release controls

16.1 Repository organization and interfaces

Use an isolated experiment package with typed contracts, environment adapters, representation models, utility heads, planners, data loaders, evaluators and tests. Connect to the current repository through a narrow `FrozenHistoryEncoder` protocol, rather than importing unverified native internals throughout the experiment. A later integration pull request should list which source files change, why they change and how the ordinary language route remains covered.

The bundle includes standalone schemas, proposed configurations, a machine-readable experiment registry, acceptance templates and an implementation handoff. The reference package is intentionally smaller than this target architecture. Its implementation-status table identifies missing native, self-supervised and production capabilities. Do not interpret a directory name or a passing synthetic test as implementation of the entire research program.

16.2 Data and checkpoint reproducibility

A run manifest records code revision, dirty-tree state, checkpoint and tokenizer hashes, environment version, split manifest, feature schema, action registry, mask mode, seed, hardware/software versions, precision, optimization parameters, effective batch, costs and artifact hashes. Record both requested and actually executed budgets. Save atomic checkpoint files and their dependent normalizers/readouts together.

Deterministic seeds are useful, but bitwise equivalence across different devices or kernel versions should not be promised. Pin the tested environment and quantify tolerances. Re-run an independent reproduction before an important promotion. The supplied validation results describe this container's CPU environment only.

16.3 Safety boundaries for learned utility

Tool permission and irreversible-action checks occur before learned scoring. The model cannot authorize file deletion, payment, account changes or external communication. Use mocked tools or a sandbox environment. Constraint violations are separately reported and cannot be averaged away by a high task reward.

Test malicious instructions embedded in retrieved documents, forged tool outputs, stale evidence, missing prerequisites, repeated no-op loops and candidates designed to exploit the scorer. Do not execute model-generated code without a bounded sandbox. Tool outputs are data; they must not change the source checkpoint, evaluation split or logging policy.

16.4 Release gates and honest claims

A research artifact may be released with a negative result. It may not be labeled a trained Rineya world model without a pinned native checkpoint evaluation. A model card should state training data, intended tasks, failure modes, prohibited actions, calibration limits and whether goals are in-span or extrapolative. A dataset card should describe labels, provenance, privacy, split grouping and license review.

Third-party papers and code are linked through the resource register rather than bundled indiscriminately. Verify the exact repository license and revision before copying code or downloading datasets; a paper's open-access status does not determine its software or dataset license. The ZIP contains original planning material and original reference code, not a redistribution of the cited implementations.

17 Delivery roadmap and ownership

17.1 A gated sequence, not a calendar promise

The following effort blocks are planning estimates. One person can hold several roles, but scientific decisions, data validation and implementation should still receive separate reviews. Native hardware availability, existing repository interfaces and the pilot's statistical requirements determine actual duration.

Block 1: approximately 3–5 focused work sessions. Research owner freezes the hypothesis, subject identity, environment, primary metric and scope. Engineering owner qualifies schemas, split logic, masks and snapshots. Reviewer checks that future information cannot leak. Deliverable: G0 report and a tiny end-to-end synthetic run.

Block 2: approximately 1–2 working weeks. Implement the measured-feature and scalar baselines, collect the first bounded parent set, establish candidate-oracle performance and measure laptop capture. Deliverable: G1 report, failure taxonomy, measured resource envelope and finalized statistical contract. No self-supervised codebook is required yet.

Block 3: approximately 1–2 working weeks. Add the learned continuous outcome encoder/predictor and sparse goal readout. Run B2/B3/B4 with matched data and budgets. Deliverable: G2 result and a decision to promote, repair or prefer the simpler evaluator.

Block 4: approximately 1–2 working weeks. Test nested label budgets, new goals and controlled clustering. Add direct successor targets only if needed by the temporal failure analysis. Deliverable: G3 transfer and discovery report with negative controls.

Block 5: approximately 1–2 working weeks. Expand context and dependency horizon, then introduce synchronous DAG tasks. Deliverable: G4 report including causal visibility, joint outcomes, scheduling and model-exploitation stress tests.

Block 6: approximately 1–3 working weeks, conditional. Port the successful interface to native typed-slot adapters, run acquisition/retention and hardware gates, then evaluate a distilled policy or short imagined-RL extension. Deliverable: G5/G6 qualification or an explicit reason to retain the sidecar. Visual modeling remains a later independent campaign.

These blocks are not a commitment to continuous background work. The priority is to complete a decisive comparison before expanding the architecture. The roadmap and experiment registry include explicit dependencies to prevent every idea from becoming simultaneous active work.

17.2 First implementation handoff

The first agent or engineer should read `START_HERE.md`, the architecture and data contracts, then run the reference tests. Next, inspect the current repository in read-only mode, identify the actual checkpoint loader and capture interface, and prepare an integration diff plan. Stop for clarification when the subject identity, sealed-data policy, allowed paths or native budget is unresolved.

The first substantive pull request should contain contracts, a simulator adapter, split checks, mask tests and an end-to-end measured-feature baseline—not native training, recurrence, codebooks and imagined RL all at once. A separate pull request should add frozen Rineya capture after numerical qualification. Each should include its exact tests, resource observations and limitations.

17.3 Decision log at every gate

Record what was expected, what was observed, which comparison isolates the mechanism, what is uncertain and what next action follows. Maintain rejected and deferred alternatives. Do not rewrite the original hypothesis after seeing evaluation results without labeling the new experiment. Useful stopping decisions are part of the research output.

18 Risk register and diagnostic responses

Risk	Detect it	Response
Future-information leakage	Causal perturbation and provenance tests	Block training/evaluation; rebuild inputs
Representation collapse	Variance, rank and decoder tests	Fix target/regularization before scaling
Hidden-state aliasing	Same observation, different history branches	Improve belief/history representation
Proxy exploitation	Predicted gain rises, real gain falls with search	Reduce search, add real anchors, revisit support
Goal entanglement	Preference-only goal changes physical prediction	Separate task instruction from utility weights
Rare failure merged away	Cluster audit by failure type	Increase resolution or retain explicit hazard features
Invalid action semantics	Same token, inconsistent executor behavior	Version and parameterize actions
Policy-mixed successor target	Different continuations share one target	Condition on policy/plan or label the mixture
Frozen-weight but worse system	End-to-end retention degrades	Restrict routing or revert controller
Apparent efficiency from hidden cost	Full cost ledger changes ranking	Recompare under total budget
Goal transfer outside feature span	New-property readout fails	Add grounded features or collect targeted data
Native numerical mismatch	Prefix/bulk or resume parity failure	Stop native extraction; repair mechanics

The highest-risk scientific shortcut is to call any useful latent reward predictor a novel world model. Keep the claim tied to the tested outcome variables and interventions. The highest-risk engineering shortcut is to accept a perfect validation loss without proving that the evaluator saw only deployment-available information.

19 Future extensions and research questions

19.1 Distributional consequences and reachable plans

After the deterministic linear-utility baseline succeeds, add stochastic outcomes and nonlinear preferences. Compare direct successor distributions with recursive latent rollouts and shared-scenario sampling. Evaluate joint failure, tail risk and reachability. Do not select an attractive predicted consequence unless a supported action or policy can plausibly realize it. DiSPO supplies an especially relevant conceptual reference for coupling future features to reachable behavior. [S04]

19.2 Learned action vocabularies

Manual operators provide a stable initial intervention vocabulary. A later model can discover reusable macro-actions from trajectories, with each code decoded into an executable, bounded option. Test whether learned options transfer across tasks and retain meaningful preconditions. Interpretability and compression are secondary to reliable execution. An action code that cannot be executed consistently is not a useful control primitive.

19.3 Internal compute as an environment

Once native recurrence is qualified, model the consequences of additional computation, checking, retrieval or stopping. Treat hidden-state prediction as an internal computation model, and evaluate whether it improves final correctness per actual latency. Compare recurrence against ordinary extra generation and tool use. This is where TAPS/CDE can become a downstream application rather than a duplicated controller project.

19.4 Visual version

A later visual Rineya variant can attach a frozen vision encoder and learn action-conditioned future visual features. Do not start by generating pixels or reproducing internet-scale video pretraining. V-JEPA 2, DINO-WM and V-JEPA 2.1 offer different predictive-representation inspirations; the relevant experiment is whether a compact action-conditioned model preserves task-critical visual variables at the available budget. [S36, S37, S39]

Separate visual perception failures from dynamics errors by comparing ground-truth state input, frozen visual features and learned visual adaptation. Test unseen appearances with fixed dynamics and unseen dynamics with similar appearances. A text-world success is not evidence of physical-world competence.

19.5 Cumulative skill acquisition

Investigate whether a consequence vocabulary supports adding a new goal readout without changing old behavior, and whether adding genuinely new transition knowledge can be localized in an adapter. Evaluate composition of old and new operators in one plan. This links the project to Rineya's broader aim of accumulating capabilities, but the world model does not automatically solve catastrophic forgetting.

A particularly informative future study changes only the goal, then only the dynamics, then only the observation rendering. These shifts test three different forms of reuse. Calling all three "generalization" conceals which part of the architecture actually transfers.

20 How to use the delivered resources

Read the executive decision and formal model first. Use the annotated bibliography to understand the closest precedents and the limits of each analogy. The `specs` directory is the implementation reference; `configs`, `schemas` and `templates` turn the report into explicit contracts. The `roadmap` directory records dependencies and future gates. The `reference` package runs without Rineya weights and demonstrates selected mechanics. Its own README states what it does not implement.

Validation outputs are included separately from proposed experiment results. The CPU demonstration is not a benchmark result and does not justify a claim that learned consequence semantics, clustering, long-context planning or skill retention have been solved. It is a reproducible starting point for detecting mistakes before native experiments become expensive.

The program's strongest claim, should later evidence support it, is precise: **under a frozen data, compute and preservation budget, a learned consequence representation improved held-out goal-conditioned action selection or reduced the required grounding labels compared with a matched scalar evaluator.** That is worth investigating. It is also specific enough to fail, be refined and ultimately produce useful knowledge.

21 Glossary and notation

Term	Meaning in this bundle
H_t, history	Observations and actions available at decision time
b_t, belief	Encoded history; not necessarily true environment state
a_t, action	Executable operator plus arguments, budget and version
ϕ_t, consequence	Measured or learned description of a transition
z_t, latent	Learned consequence coordinates without assumed named axes
g, goal	Task/preference specification; roles recorded separately
w_g, utility weights	Readout for an explicitly represented feature family
ψ^π, successor features	Expected discounted consequences under policy π
A, candidate plan	Precommitted actions or an explicit conditional policy tree
Counterfactual family	Branches, goals and variants from the same underlying parent
Grounding	Independent task feedback connecting representations to utility
In-span revaluation	Changing utility over already represented consequences
Native qualification	Tests using the exact selected Rineya artifact and runtime
Promotion	Meeting predeclared scientific, preservation and cost gates

22 Delivered reference mechanics and validation

22.1 What was actually run

The bundle's original CPU package was tested in the recorded environment: Python 3.13.5, PyTorch 2.10.0+cpu and NumPy 2.3.5. **60 tests passed.** They cover finite-horizon consequence accounting, goal revaluation, mask conventions, future-action invariance, parent grouping, train-only codebook fitting, schema/temporal checks, simulator behavior, tensor shapes, finite gradients and checkpoint roundtrip. Full test names and logs are included. No GPU or native Rineya checkpoint was used.

A 40,325-parameter causal action-token Transformer was trained for 250 updates on an original KeyDoor toy dataset. The check-set mean-squared feature error changed from 0.540924 to 0.002186. This is a mechanics demonstration with repeated support in a tiny state space, not an independent generalization benchmark. Its learned bottleneck is supervised through measured features; it does not validate self-supervised semantic discovery or clustering.

22.2 An important observed limitation

The average error alone would be misleading. On the illustrated safe sequence `take_key -> open_door -> collect`, the model predicted a cumulative object-gained feature of approximately **0.064**, although the exact simulator value was **1.000**. Under the damage-averse goal, it scored this route at approximately **-3.46**, although the exact value was **7.00**. The demo therefore does **not** pass a strong per-plan fidelity gate merely because its aggregate error fell.

This failure is deliberately preserved in the delivered evidence. It illustrates why the proposed program requires prerequisite-stratified checks, candidate regret and independent real outcomes, rather than only an average reconstruction score. The code's mechanical tests passing means the selected tested invariants held; it does not mean its learned predictions are accurate enough for deployment. A later coverage or training change should be a separately logged experiment, not a silent rewrite of this run.

22.3 Resource and artifact checks

Three JSON schemas, four illustrative trajectory records, seven proposed configurations and the 43-entry source register were checked by the bundled validation script. Temporal and feature-dimension checks extend the JSON Schema validation for the initial serial format. Production provenance, native memory, stochastic calibration, parallel execution and retention remain unmeasured.

The final archive includes a file manifest with SHA-256 hashes. The manifest excludes itself to avoid a recursive hash definition. The verification utility checks each listed file after extraction. These integrity checks are reproducibility aids, not a security certification or validation of scientific claims.

22.4 Issues found during construction

An initial unit-test run exposed a Python/PyTorch naming collision: a submodule named `type` conflicted with the parent module's dtype-conversion method. Renaming it to `type_embedding` fixed the forward path, after which the suite was rerun. This is recorded as a resolved engineering issue, not hidden behind the final passing result.

The report and machine-readable status files distinguish implemented reference mechanics from planned native work. The most important next implementation task is still the matched scalar/feature baseline and qualified Rineya capture, not an expansion into imagined RL.

23 Source register

Sources accessed 5 October 2026. Full annotations and review-depth labels are in `research/ANNOTATED_READING.md`; machine-readable metadata and BibTeX are included. Papers are evidence for their reported settings, not measured Rineya outcomes.

[S01] Mastering diverse control tasks through world models. Hafner, Danijar; Pasukonis, Jurgis; Ba, Jimmy; Lillicrap, Timothy. 2025; Nature. https://www.nature.com/articles/s41586-025-08744-2

[S02] Successor Features for Transfer in Reinforcement Learning. Barreto, Andre; others. 2017; NeurIPS. https://arxiv.org/abs/1606.05312

[S03] Universal Successor Features Approximators. Borsa, Diana; others. 2019; ICLR. https://arxiv.org/abs/1812.07626

[S04] Distributional Successor Features Enable Zero-Shot Policy Optimization. Zhu, Chuning; Wang, Xinqi; Han, Tyler; Du, Simon S.; Gupta, Abhishek. 2024; NeurIPS; arXiv v4 2025. https://arxiv.org/abs/2403.06328

[S05] Successor Features for Efficient Multisubject Controlled Text Generation. Cao, Meng; Fatemi, Mehdi; Cheung, Jackie Chi Kit; Shabanian, Samira. 2023; arXiv preprint. https://arxiv.org/abs/2311.04921

[S06] Decision Transformer: Reinforcement Learning via Sequence Modeling. Chen, Lili; others. 2021; NeurIPS. https://arxiv.org/abs/2106.01345

[S07] Offline Reinforcement Learning as One Big Sequence Modeling Problem. Janner, Michael; Li, Qiyang; Levine, Sergey. 2021; NeurIPS; author project. https://trajectory-transformer.github.io/

[S08] UniMASK: Unified Inference in Sequential Decision Problems. Carroll, Micah; others. 2022; NeurIPS. https://arxiv.org/abs/2211.10869

[S09] Masked Trajectory Models for Prediction, Representation, and Control. Wu, Philipp; Majumdar, Arjun; Stone, Kevin; Lin, Yixin; Mordatch, Igor; Abbeel, Pieter; Rajeswaran, Aravind. 2023; ICML / PMLR. https://proceedings.mlr.press/v202/wu23o.html

[S10] Learning Action Representations for Reinforcement Learning. Chandak, Yash; Theocharous, Georgios; Kostas, James; Jordan, Scott; Thomas, Philip. 2019; ICML / PMLR. https://proceedings.mlr.press/v97/chandak19a.html

[S11] Mastering Atari, Go, chess and shogi by planning with a learned model. Schrittwieser, Julian; others. 2020; Nature; primary arXiv version. https://arxiv.org/abs/1911.08265

[S12] DeepMDP: Learning Continuous Latent Space Models for Representation Learning. Gelada, Carles; others. 2019; ICML / PMLR. https://proceedings.mlr.press/v97/gelada19a.html

[S13] Learning Invariant Representations for Reinforcement Learning without Reconstruction. Zhang, Amy; McAllister, Rowan; Calandra, Roberto; Gal, Yarin; Levine, Sergey. 2021; ICLR. https://arxiv.org/abs/2006.10742

[S14] Representation Learning with Contrastive Predictive Coding. van den Oord, Aaron; Li, Yazhe; Vinyals, Oriol. 2018; arXiv preprint. https://arxiv.org/abs/1807.03748

[S15] Neural Discrete Representation Learning. van den Oord, Aaron; Vinyals, Oriol; Kavukcuoglu, Koray. 2017; NeurIPS. https://arxiv.org/abs/1711.00937

[S16] VICReg: Variance-Invariance-Covariance Regularization for Self-Supervised Learning. Bardes, Adrien; Ponce, Jean; LeCun, Yann. 2022; ICLR. https://arxiv.org/abs/2105.04906

[S17] Diversity is All You Need: Learning Skills without a Reward Function. Eysenbach, Benjamin; others. 2019; ICLR. https://arxiv.org/abs/1802.06070

[S18] Generalized Decision Transformer for Offline Hindsight Information Matching. Furuta, Hiroki; Matsuo, Yutaka; Gu, Shixiang Shane. 2021; arXiv preprint. https://arxiv.org/abs/2111.10364

[S19] RUDDER: Return Decomposition for Delayed Rewards. Arjona-Medina, Jose A.; others. 2019; NeurIPS. https://arxiv.org/abs/1806.07857

[S20] Deep Reinforcement Learning from Human Preferences. Christiano, Paul F.; others. 2017; NeurIPS. https://arxiv.org/abs/1706.03741

[S21] Let's Verify Step by Step. Lightman, Hunter; others. 2023; arXiv preprint. https://arxiv.org/abs/2305.20050

[S22] When to Trust Your Model: Model-Based Policy Optimization. Janner, Michael; Fu, Justin; Zhang, Marvin; Levine, Sergey. 2019; NeurIPS. https://arxiv.org/abs/1906.08253

[S23] Conservative Q-Learning for Offline Reinforcement Learning. Kumar, Aviral; others. 2020; NeurIPS. https://arxiv.org/abs/2006.04779

[S24] A Distributional Perspective on Reinforcement Learning. Bellemare, Marc G.; Dabney, Will; Munos, Remi. 2017; ICML. https://arxiv.org/abs/1707.06887

[S25] Simple and Scalable Predictive Uncertainty Estimation using Deep Ensembles. Lakshminarayanan, Balaji; Pritzel, Alexander; Blundell, Charles. 2017; NeurIPS. https://arxiv.org/abs/1612.01474

[S26] Scaling Laws for Reward Model Overoptimization. Gao, Leo; Schulman, John; Hilton, Jacob. 2022; arXiv preprint. https://arxiv.org/abs/2210.10760

[S27] LoRA: Low-Rank Adaptation of Large Language Models. Hu, Edward J.; others. 2022; ICLR. https://arxiv.org/abs/2106.09685

[S28] QLoRA: Efficient Finetuning of Quantized LLMs. Dettmers, Tim; others. 2023; NeurIPS. https://arxiv.org/abs/2305.14314

[S29] RULER: What's the Real Context Size of Your Long-Context Language Models?. Hsieh, Cheng-Ping; others. 2024; COLM. https://arxiv.org/abs/2404.06654

[S30] LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding. Bai, Yushi; others. 2024; ACL. https://arxiv.org/abs/2308.14508

[S31] TextWorld: A Learning Environment for Text-based Games. Cote, Marc-Alexandre; others. 2018; arXiv preprint. https://arxiv.org/abs/1806.11532

[S32] ALFWorld: Aligning Text and Embodied Environments for Interactive Learning. Shridhar, Mohit; others. 2021; ICLR. https://arxiv.org/abs/2010.03768

[S33] WebShop: Towards Scalable Real-World Web Interaction with Grounded Language Agents. Yao, Shunyu; others. 2022; NeurIPS. https://arxiv.org/abs/2207.01206

[S34] Deep Reinforcement Learning at the Edge of the Statistical Precipice. Agarwal, Rishabh; others. 2021; NeurIPS. https://arxiv.org/abs/2108.13264

[S35] Hindsight Experience Replay. Andrychowicz, Marcin; others. 2017; NeurIPS. https://arxiv.org/abs/1707.01495

[S36] V-JEPA 2: Self-Supervised Video Models Enable Understanding, Prediction and Planning. Assran, Mido; others. 2025; arXiv preprint. https://arxiv.org/abs/2506.09985

[S37] DINO-WM: World Models on Pre-trained Visual Features enable Zero-shot Planning. Zhou, Gaoyue; Pan, Hengkai; LeCun, Yann; Pinto, Lerrel. 2024; arXiv preprint. https://arxiv.org/abs/2411.04983

[S38] From Word to World: Can Large Language Models be Implicit Text-based World Models?. Li, Yixia; others. 2025; arXiv preprint; revision 2026. https://arxiv.org/abs/2512.18832

[S39] V-JEPA 2.1: Unlocking Dense Features in Video Self-Supervised Learning. Mur-Labadia, Lorenzo; others. 2026; arXiv preprint. https://arxiv.org/abs/2603.14482

[S40] Attention Is All You Need. Vaswani, Ashish; others. 2017; NeurIPS. https://arxiv.org/abs/1706.03762

[S41] torch.nn.Transformer documentation. {PyTorch contributors}. 2026; official documentation; mutable. https://docs.pytorch.org/docs/2.10/generated/torch.nn.Transformer.html

[S42] Qwen3 model documentation. {Hugging Face contributors}. 2026; official documentation; mutable. https://huggingface.co/docs/transformers/model_doc/qwen3

[S43] Gymnasium Env API. {Farama Foundation contributors}. 2026; official documentation; mutable. https://gymnasium.farama.org/api/env/

23.1 Historical internal references

[I01] Rineya Intelligence Research Handbook, 30 September 2026, study edition 1.0. Personal Library report; chapter 1, source identity and laptop capture mechanics. The exact source checkpoint was reported as 596,049,920 parameters; current identity and qualification must be rechecked. Not a fresh repository audit.

[I02] Rineya: Architectures for Composable Skills, 21 September 2026. Personal Library report; sections 12–13 on paired evaluation, preservation, compute reporting and sealed-data discipline. These are historical research records, not external validation.

[I03] The present conversation, 5 October 2026. User proposal: learn consequence/reward-related embeddings with action tokens, attention, long-range and parallel structure, and goal-conditioned evaluation. Concept origin; not empirical evidence.